



UMAG
Universidad de Magallanes

Lenguaje C

Integrante:
Martina Márquez

Fecha: 20 de septiembre de 2026

Profesora: Jacqueline Aldridge
Asignatura: Estructuras de Datos

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivos Específicos	2
3. Punteros	3
3.1. Descripción del trabajo	3
3.2. Procedimiento	4
3.2.1. Paso 1: "Declaración de variables y punteros"	4
3.2.2. Malloc	4
3.2.3. Direcciones iniciales	4
3.2.4. Modificaciones y sobrescritura	5
3.2.5. Valores en el arreglo	5
3.2.6. Reasignación	5
3.2.7. Consolidación de valores y salida por consola	5
3.3. Resultados	6
4. Estructura y Makefile	7
4.1. Descripción del trabajo	7
4.2. Procedimiento	7
4.2.1. Organización de directorios	7
4.2.2. Desarrollo	7
4.3. Resultados	7
5. Conclusiones	8

1. Introducción

El presente trabajo se trata del repaso de punteros e implementación un sistema de gestión de inventario en lenguaje C para diversos artículos, tomando como base la estructura modular revisada en la ayudantía. El código fuente se encuentra organizado en directorios separados y permite persistir la información en un archivo de formato CSV. Asimismo, el proceso de compilación se automatiza a través de un Makefile y la correcta administración de la memoria dinámica es verificada con la herramienta Valgrind, garantizando la inexistencia de fugas de memoria.

2. Objetivos

Documentar mediante un informe en LaTeX las practicas con memoria, punteros y diseño de estructuras en C evidenciando su implementacion en un sistema funcional

2.1. Objetivos Específicos

- Explicar de forma gráfica y secuencial el comportamiento de variables y punteros deduciendo la salida del programa a partir del estado de la memoria.
- Detallar la creación de la estructura "Producto", los métodos de generación aleatoria/ingreso de datos y la exportación hacia un archivo CSV.
- Describir la organización del proyecto en directorios (src/ e inc/), la automatización de la compilación mediante Makefile y la verificación de ausencia de fugas de memoria utilizando valgrind.

3. Punteros

3.1. Descripción del trabajo

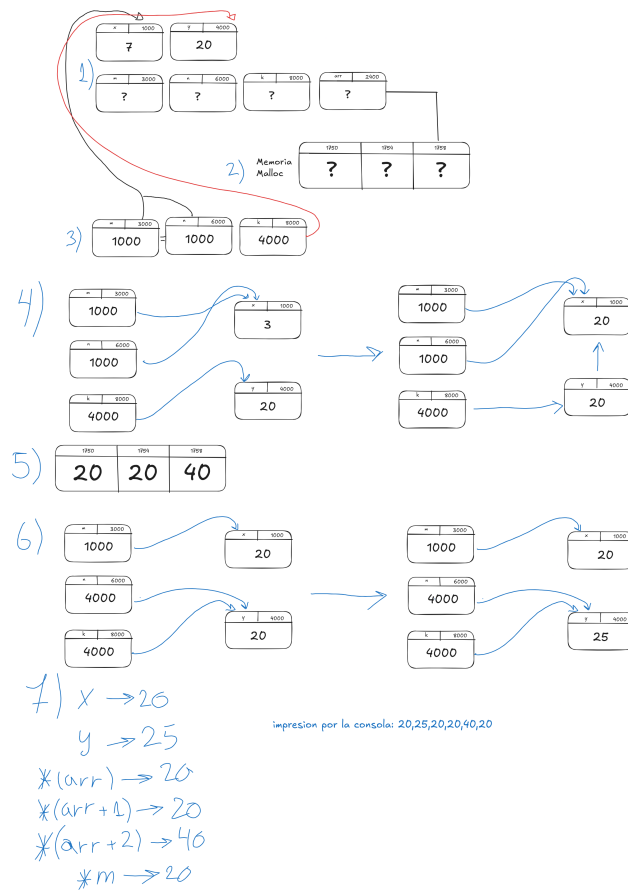
La metodología empleada consiste en representar gráficamente el estado de la memoria RAM modelando cada variable, puntero y bloque de asignación dinámica como una celda o «caja» con su respectiva dirección y contenido. A través de un seguimiento secuencial línea por línea, se registran los cambios de valor y las reasignaciones de referencias, permitiendo deducir y verificar analíticamente la salida final generada por la función `printf` a partir del estado gráfico resultante de la memoria.

El programa en lenguaje C sujeto a análisis se presenta en el Listado 1:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 3
5
6 int main() {
7     int x = 7, y = 20, *m, *n, *k;
8     int *arr;
9
10    arr = (int*) malloc(MAX * sizeof(int));
11
12    m = &x;
13    n = m;
14    k = &y;
15
16    *m = 3;
17    *n = *k;
18
19    *(arr) = *m;
20    *(arr + 1) = *k;
21    *(arr + 2) = x + y;
22
23    n = &y;
24    *n += 5;
25
26    printf("%d, %d, %d, %d, %d, %d\n", x, y, *arr, *(arr + 1), *(arr + 2), *m);
27
28    return 0;
29 }
```

Listing 1: Código fuente analizado para el seguimiento de punteros

3.2. Procedimiento



3.2.1. Paso 1: "Declaración de variables y punteros"

Se dibuja el recuadro de la declaración de las variables y los punteros en la memoria. La variable `x` se aloja en la dirección 1000 con valor inicial 7, mientras que la variable `y` se ubica en la dirección 4000 con el valor 20. Debajo de ellas se presentan los punteros `m` (en 3000), `n` (en 6000), `k` (en 8000) y `arr` (en 2400), todos conteniendo un signo de interrogación (?), lo que denota que aún no han sido inicializados y almacenan valores indeterminados.

3.2.2. Malloc

Se ilustra la memoria dinámica reservada para el arreglo `arr`, conformada por un bloque continuo de tres celdas de memoria identificadas con las direcciones 1750, 1754 y 1758. Como la memoria acaba de ser solicitada mediante `malloc` sin inicializar sus valores, las tres posiciones contienen basura de memoria (?).

3.2.3. Direcciones iniciales

Los punteros reciben las primeras referencias de memoria. El puntero `m` almacena la dirección 1000 correspondiente a la variable `x`, y el puntero `n` copia dicho valor (1000), por lo que ambos quedan enlazados hacia `x`. A su vez, el puntero `k` almacena la dirección 4000, apuntando directamente a la variable `y`. Las flechas ilustran cómo estas referencias conectan cada puntero con la celda de la variable respectiva.

3.2.4. Modificaciones y sobreescritura

Este paso se divide en dos momentos. En el primer esquema, tras la instrucción $*m = 3$, se desreferencia m y se modifica la celda de la variable x (dirección 1000), la cual pasa de valer 7 a valer 3. En el segundo esquema, mediante la instrucción $*n = *k$, se toma el valor contenido en la celda apuntada por k (la variable y , que almacena 20) y se copia hacia la celda apuntada por n (la variable x), provocando que x cambie su valor de 3 a 20.

3.2.5. Valores en el arreglo

Se actualizan las tres celdas del bloque de memoria (1750, 1754 y 1758). En la primera posición (1750) se almacena el valor 20 proveniente de $*m$; en la segunda posición (1754) se guarda 20 obtenido a través de $*k$; y en la tercera celda (1758) se guarda el valor 40, correspondiente al resultado de evaluar la suma aritmética de las variables actuales $x + y$ ($20 + 20$).

3.2.6. Reasignación

El seguimiento muestra la reasignación del puntero n y el incremento posterior. Primero, n deja de apuntar a x y pasa a almacenar la dirección 4000 para referenciar a la variable y , quedando tanto n como k apuntando a dicha variable. En el esquema contiguo, la instrucción $*n += 5$ suma 5 unidades de forma indirecta sobre la variable y , actualizando su contenido de 20 a 25, mientras que x permanece inalterada con su valor de 20.

3.2.7. Consolidación de valores y salida por consola

En la esquina inferior se sintetiza el estado final de todos los términos evaluados antes del término del programa: x termina en 20, y en 25, las posiciones del arreglo $*(arr)$, $*(arr+1)$ y $*(arr+2)$ quedan en 20, 20 y 40 respectivamente, y la desreferenciación $*m$ arroja 20. Con base en estos valores consolidados, se concluye que la cadena final impresa por la consola corresponde exactamente a 20,25,20,20,40,20.

3.3. Resultados

Tras completar el análisis gráfico y el seguimiento secuencial de las instrucciones en memoria, se determinó con precisión el estado final de cada celda justo antes del término del programa. En primer lugar, la variable escalar `x` finalizó con un valor de 20. Dicha variable fue declarada inicialmente en 7, mutó de forma indirecta a 3 mediante la instrucción `*m = 3`, y finalmente fue sobrescrita con un 20 a través de `*n = *k`, instrucción que copió el valor contenido en la variable apuntada por `k`. Por otro lado, la variable `y` alcanzó un valor final de 25. Aunque comenzó en 20 y conservó esa cifra durante la mayor parte de la rutina, la reasignación del puntero `n` hacia su dirección (`n = &y`) permitió que la operación `*n += 5` incrementara su valor en 5 unidades de forma indirecta. En lo que respecta al arreglo dinámico `arr`, cada una de sus tres posiciones contiguas almacenó valores determinados mediante aritmética de punteros. La primera celda, correspondiente a la dirección base 1750 y referenciada como `*arr`, guardó el valor 20 asignado desde `*m`. La segunda celda, ubicada en la dirección 1754 y accedida mediante `*(arr + 1)`, almacenó un 20 transferido a través de `*k`. En la tercera celda, situada en la dirección 1758 bajo la expresión `*(arr + 2)`, se registró un valor de 40 como resultado de evaluar la suma de las variables `x + y` ($20 + 20$). Finalmente, el puntero desreferenciado `*m` arrojó un valor de 20, ya que conservó intacta la referencia a la dirección de memoria de `x` a lo largo de toda la ejecución del código.

Cuadro 1: Estado final de las variables, celdas y punteros tras la ejecución

Identificador Evaluado	Dirección Asociada	Valor Final	Origen del Valor
<code>x</code>	1000	20	Sobrescrita por <code>*n = *k</code>
<code>y</code>	4000	25	Incremento directo por <code>*n += 5</code>
<code>*arr</code>	1750	20	Copia de <code>*m</code> (<code>x</code>)
<code>*(arr + 1)</code>	1754	20	Copia de <code>*k</code> (<code>y</code>)
<code>*(arr + 2)</code>	1758	40	Suma de <code>x + y</code> ($20 + 20$)
<code>*m</code>	1000	20	Desreferenciación de <code>x</code>

4. Estructura y Makefile

4.1. Descripción del trabajo

El propósito de esta actividad consistió en diseñar e implementar un sistema básico de inventario para una empresa empleando tipos de datos estructurados en C. El programa contiene una estructura denominada **Producto**, compuesta por los campos nombre, cantidad, proveedor y precio. La instanciación de los registros se automatizó combinando nombres predefinidos desde un arreglo de cadenas, cantidades numéricas generadas de forma aleatoria, proveedores asignados mediante cadenas de números y letras aleatorias de cinco caracteres y precios ingresados por el usuario a través de consola. Todos los registros generados se almacenan en un archivo con formato CSV, incorporando su respectiva fila de encabezado y separadores por comas para asegurar su portabilidad y lectura. Además, el proyecto se estructuró de manera modular separando cabeceras e implementaciones, automatizando el ciclo de compilación y ejecución con un archivo **Makefile**, y asegurando la ausencia total de fugas de memoria en la ejecución con la herramienta **valgrind**.

4.2. Procedimiento

4.2.1. Organización de directorios

El código fuente y las cabeceras se organizaron respetando las reglas para proyectos en C. En el directorio `inc/` se incorporó el archivo de cabecera `producto.h`. Por otra parte, la carpeta `src/` reúne el código ejecutable, conteniendo tanto a `producto.c`, encargado de implementar la lógica interna y la manipulación de la estructura, como a `main.c`, que actúa como el punto de entrada y controlador principal del inventario. Adicionalmente, se configuraron dos carpetas auxiliares para aislar los productos intermedios y finales del proceso de compilación. El directorio `obj/` se estableció como destino para almacenar los archivos objeto generados tras compilar individualmente cada unidad de traducción. Por último, el directorio `build/` se reservó para recibir los artefactos resultantes de la ejecución.

4.2.2. Desarrollo

El desarrollo del inventario se implementó en C, separando cabeceras en `inc/`, código ejecutable en `src/`, y aislando los objetos compilados y el binario final en `obj/` y `build/` mediante un archivo **Makefile**. En `inc/producto.h` se definió la estructura **Producto** con los campos nombre, cantidad, proveedor y precio, junto a los prototipos de las funciones principales. En `src/producto.c`, la función `crear_producto()` asigna memoria dinámica con `malloc` para las cadenas de texto, copia sus datos con `strcpy` y valida los valores numéricos. Para evitar fugas de memoria, la función `eliminar_producto()` libera estos recursos mediante llamadas a `free`. Por último, en `src/main.c` se instancian cinco productos combinando nombres de periféricos predefinidos, cantidades generadas al azar con `rand()`, un código alfanumérico aleatorio de cinco caracteres para el proveedor y precios validados desde el teclado con `scanf`. Luego de mostrar el inventario en pantalla con `print_producto()`, la función `productos_to_csv()` guarda los registros separados por comas con su encabezado en `build/inventario.csv`, finalizando con la liberación de toda la memoria dinámica.

4.3. Resultados

La ejecución del programa confirmó el funcionamiento correcto de todos los segmentos. Al ejecutar **make run**, el programa solicitó y validó con éxito el precio por teclado para cada uno de los cinco artículos, generando de forma automática sus cantidades y códigos de proveedor antes de imprimir el resumen por pantalla. Asimismo, la función `productos_to_csv()` creó correctamente el archivo `build/inventario.csv` con su encabezado y registros separados por comas. Finalmente, la verificación con **valgrind** confirmó cero fugas de memoria, demostrando que `eliminar_producto()` liberó adecuadamente toda la memoria reservada con `malloc`.

5. Conclusiones

El desarrollo conjunto de ambas actividades permitió consolidar los fundamentos de la administración de memoria, repasar lo aprendido y aprender cosas nuevas en lenguaje C. En la primera etapa, el seguimiento gráfico de punteros demostró cómo se distribuyen y mutan los datos en la memoria física, dejando en claro que saber cómo apuntan y modifican datos las variables permite anticipar el resultado del código con total seguridad. En la segunda etapa, esta base sirvió para armar una estructura que combina diferentes variables y reserva memoria con malloc, persistencia de datos en formato CSV y automatización de la compilación mediante un Makefile. La validación del código con herramientas como valgrind confirmó que una estructura ordenada en carpetas y archivos no solo facilita el mantenimiento del proyecto, sino que es fundamental para asegurarse de liberar la memoria a tiempo, y prevenir fugas de memoria. En síntesis, el dominio de estos conceptos sienta una buena base para programar estructuras de datos más complejas de manera segura.